

Iteration 2 Worksheet - Group 6

Paying off technical debt

1. Postponement of “Look for Available Items” to Iteration 2

We had to spend more time in this iteration working on a feature that we initially planned to have ready earlier. Here is the code for [HomeActivity.java](#) and [ItemActivity.java](#) where its implemented.

I'd classify this as deliberate and prudent because we understood the tradeoff of pushing only completed features at the time. We knew it required us to spend more time in this iteration2 and were okay with it.

2. Incomplete Unit Testing

The test code was written before the final changes were made to the logic layer, causing it to fall below the 80% test coverage threshold. Due to time constraints, we were unable to meet the standard, which resulted in lost points and the need for refactoring in iteration2.

[ItemService.java](#) and [UserProfileService.java](#) were the 2 classes that had to be updated.

I'd classify this as inadvertent and prudent because we now have a better understanding of the correct process order. We also recognize the importance of actively communicating changes to minimize the impact on existing code.

SOLID

Commit: [c9da7960](#) - QuizService.java

Issue: [#128](#)

Dependency Inversion Violation:

The QuizService class creates an instance of AccessFlashcards directly in its constructor. This means it depends on a concrete class rather than an abstract interface. If you need to change how flashcards are accessed, you'll have to modify QuizService itself, making the code less flexible and harder to test.

Single Responsibility Violation:

QuizService handles multiple tasks—it manages the quiz flow (tracking questions, recording answers, navigating between flashcards) and also calculates scores and saves quiz results. By mixing these responsibilities in one class, any change in one area could affect the others. Separating these tasks into different classes would simplify the design and make the code easier to maintain.

Retrospective

In our iteration one retrospective, we decided to define more specific roles for each team member to ensure effective contributions. As a result, everyone had the opportunity to create their own branches to work on their respective [developer tasks](#). We also agreed to communicate major changes before committing and continued maintaining detailed [meeting notes](#), documenting significant changes and plans for each iteration.

1. Start : What should we try in the next phase ?

- Hold group meetings as early as possible to plan for the next iteration.
- Minimize merge conflicts as much as possible.
- Ensure the developer branch is updated as a backup of the main branch at least one to two days before the iteration deadline.

2. Stop : What should we avoid in the next phase ?

- Not responding to messages promptly.
- Not pushing to develop the branch at least one to two days before the iteration deadline.

3. Continue : What should we keep doing in the next phase ?

- Holding regular TR meetings on Discord.
- Maintaining a structured approach to work instead of rushing tasks.
- Using GitFlow as our branching strategy.
- Supporting group members who are struggling with a task.
- Speaking up, when a member is struggling with a task.
- Assigning specific development task roles.

Design patterns

1. Design Pattern Name: Template Method Pattern

The **Template Method Pattern** is a behavioral design pattern that defines a skeleton for an algorithm in a **base class** while allowing **subclasses to override specific steps** without changing the overall structure.

In our project, the **BaselItemAdapter** class follows this pattern by defining a general structure for displaying items in a **RecyclerView**, while **ItemAdapter** and **RelatedItemsAdapter** customize its behavior for different use cases.

2. Where is This Pattern Used?

The pattern is implemented in the following files:

- **BaselItemAdapter.java** (Defines the template for item display)
[BaselItemAdapter.java](#)
 - **ItemAdapter.java** (Customizes behavior for standard items)
[ItemAdapter.java](#)
 - **RelatedItemsAdapter.java** (Handles related items, following the same structure)
[RelatedItemsAdapter.java](#)
-

3. Explanation of How It Works

The **BaselItemAdapter** class provides a **template structure** for handling items in a **RecyclerView**, but leaves specific implementation details to subclasses.

- **BaselItemAdapter.java (Template Class)**
 - Defines the core logic for managing **RecyclerView** items.
 - Implements **onCreateViewHolder()** and **onBindViewHolder()** to enforce a consistent display structure.
 - Declares an **abstract method onItemClick(Item item)**, which subclasses must override.
- **ItemAdapter.java & RelatedItemsAdapter.java (Concrete Implementations)**

- Extend `BaseItemAdapter` and override `onItemClick()` to define specific behaviors.
- `ItemAdapter` launches `ItemActivity` when an item is clicked.
- `RelatedItemsAdapter` handles a list of related items.

Why This is the Template Method Pattern?

1. The `BaseItemAdapter` defines the **overall algorithm structure**.
2. **Subclasses** (`ItemAdapter`, `RelatedItemsAdapter`) **override only specific steps**, following the pattern.
3. **Code reusability and consistency** are maintained across different item types.

Iteration 1 Feedback fixes

Issue: [#50 - Non-functional buttons \(Saved, Bookmark, and FAQ\)](#)

The issue was that the **Saved, Bookmark, and FAQ** buttons in the navigation were non-functional.

Fixes Implemented:

1. Implemented Saved & Bookmark Feature

These were part of the [Look for Available Items](#) feature, specifically the [User Story: Bookmark and Save Items](#).

- **Added Fragments (For You & Saved)** → [f4675c4d](#)
 - [ForYouFragment](#)
 - [SavedFragment](#)
 - [HomeActivity](#)
- **Item Activity Enhancements** → [9c48a9f7](#)

2. Implemented FAQ Button (Temporary Placeholder)

Since the FAQ section is a [separate feature](#), we added a placeholder page indicating "Future Implementation".

- **NavigationActivity** → [f7e1c10e](#)
- **FaqActivity** → [f7e1c10e](#)